

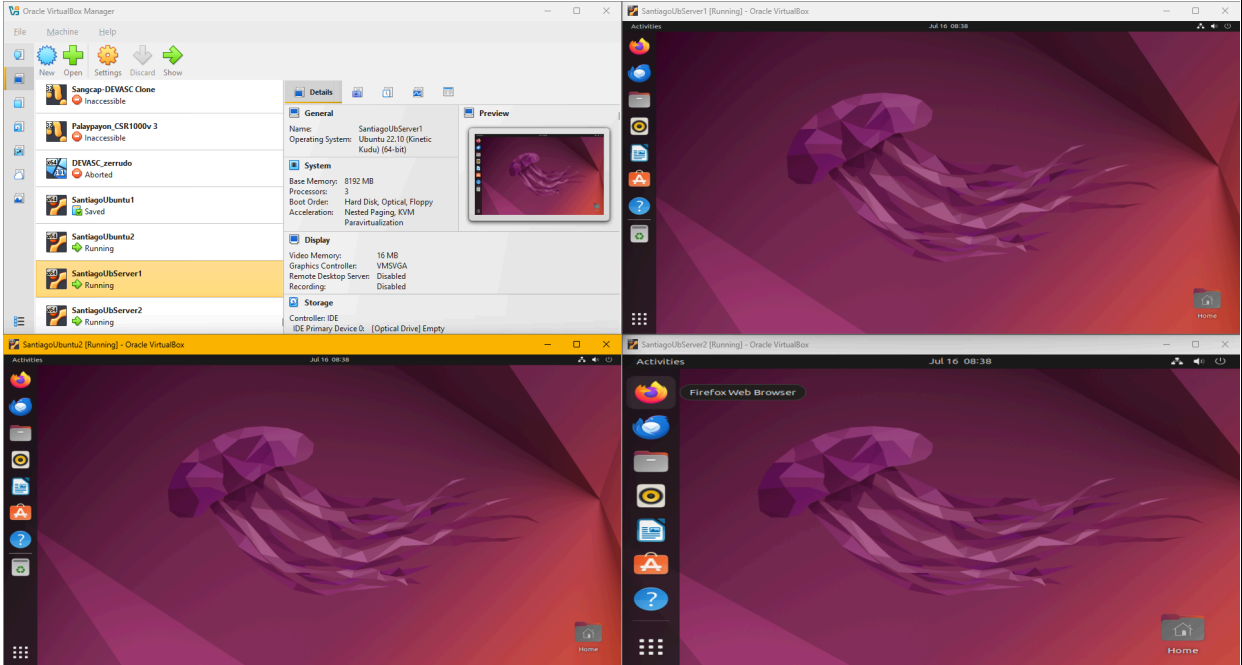
COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	07-16-26	DATE SUBMITTED:	07-16-26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 2.1

Hands-on Activity - SSH Key-Based Authentication and GIT Setup

1. PROCEDURE OUTPUT

Completion of Activity 1.1: 3 Virtual Machines:



Task 1: Create an SSH Key Pair for User Authentication

1.

The simplest way to generate a key pair is to run *ssh-keygen* without arguments. In this case, it will prompt for the file in which to store keys. First, the tool asked where to save the file. SSH keys for user authentication are usually stored in the users .ssh directory under the home directory. However, in enterprise environments, the location is often different. The default key file name depends on the algorithm, in this case *id_rsa* when using the default RSA algorithm. It could also be, for example, *id_dsa* or *id_ecdsa*.

2.

Issue the command *ssh-keygen -t rsa -b 4096*. The algorithm is selected using the -t option and key size using the -b option.

```
workstation@workstation:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/workstation/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/workstation/.ssh/id_rsa
Your public key has been saved in /home/workstation/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:a1Lc56GVdyiyvciHPN3bbtMeRDM21ICZrJRWPBNNaY workstation@workstation
The key's randomart image is:
+---[RSA 4096]---+
|          o*+Bo  |
|       .+.0...o  |
|      oE. o * .  |
|     . . .+.o    |
|    S o * oo.    |
|   . . 0 +. .   |
|  . o.ooo. . .  |
|   o .+.o..o+   |
|    oo. .==     |
+-----[SHA256]-----+
```

3. When asked for a passphrase, just press enter. The passphrase is used for encrypting the key, so that it cannot be used even if someone obtains the private key file. The passphrase should be cryptographically strong.
4. Verify that you have created the key by issuing the command `ls -la .ssh`. The command should show the .ssh directory containing a pair of keys. For example, id_rsa.pub and id_rsa.

```
workstation@workstation:~$ ls -la .ssh
total 24
drwx----- 2 workstation workstation 4096 Jul 16 09:09 .
drwxr-x--- 15 workstation workstation 4096 Jul 16 09:00 ..
-rw----- 1 workstation workstation 3389 Jul 16 09:09 id_rsa
-rw-r--r-- 1 workstation workstation 749 Jul 16 09:09 id_rsa.pub
-rw----- 1 workstation workstation 2098 Jul 16 09:08 known_hosts
-rw----- 1 workstation workstation 1262 Jul 16 09:08 known_hosts.old
```

Task 2: Copying the Public Key to the remote servers

1. To use public key authentication, the public key must be copied to a server and installed in an `authorized_keys` file. This can be conveniently done using the `ssh-copy-id` tool.
2. Issue the command similar to this: `ssh-copy-id -i ~/.ssh/id_rsa user@host`
3. Once the public key has been configured on the server, the server will allow any connecting user that has the private key to log in. During the login process, the client proves possession of the private key by digitally signing the key exchange.

```
workstation@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa server1@server1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/workstation
/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompt
ed now it is to install the new keys
server1@server1's password:
```

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'server1@server1'"
and check to make sure that only the key(s) you wanted were added.

```
workstation@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa santiagoserver2@server2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/workstation
/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompt
ed now it is to install the new keys
santiagoserver2@server2's password:
```

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'santiagoserver2@server2'"
and check to make sure that only the key(s) you wanted were added.

4. On the local machine, verify that you can SSH with Server 1 and Server 2. What did you notice? Did the connection ask for a password? If not, why?

```
workstation@workstation:~$ ssh server1@server1
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-134-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Jul 16 09:07:20 2026 from 192.168.56.113
server1@server1:~$
logout
Connection to server1 closed.
workstation@workstation:~$ ssh santiagoserver2@server2
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.19.0-32-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

13 updates can be applied immediately.
4 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
```

The connection no longer asked for a password because the public key has been copied in the servers' id_rsa files. This conducts a pair check by comparing the public key saved and the public key of the connecting machine. Since they match, the connection can be established without a password prompt.

Reflections:

Answer the following:

1. How will you describe the ssh-program? What does it do?

The SSH program is excellent and secure for server management. This activity is a great demonstration of the security capabilities of SSH through the use of key-pairs. Instead of relying on unsecure passwords, we can simply set up key-pairs on a client and server to allow secure and encrypted connections with minimal security risks.

2. How do you know that you already installed the public key to the remote servers?

The simplest way to know is if the ssh connection request no longer requires a password. Simply running the command `ssh user@useraddress` is enough to establish a secure connection. This becomes automatic because the server address is saved in the `/etc/hosts` file of the host machine while the rsa public key of the host machine is saved in the server machine, allowing the secure connection.

Task 3: Set up the Git Repository

1. On the local machine, verify the version of your git using the command *which git*. If a directory of git is displayed, then you don't need to install git. Otherwise, to install git, use the following command: *sudo apt install git*

```
workstation@workstation:~$ which git
workstation@workstation:~$ sudo apt install git
[sudo] password for workstation:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcb-1.0-0 libsmbios-c2
Use 'sudo apt autoremove' to remove them.
```

2. After the installation, issue the command *which git* again. The directory of git is usually installed in this location: *user/bin/git*.

```
workstation@workstation:~$ which git
/usr/bin/git
workstation@workstation:~$
```

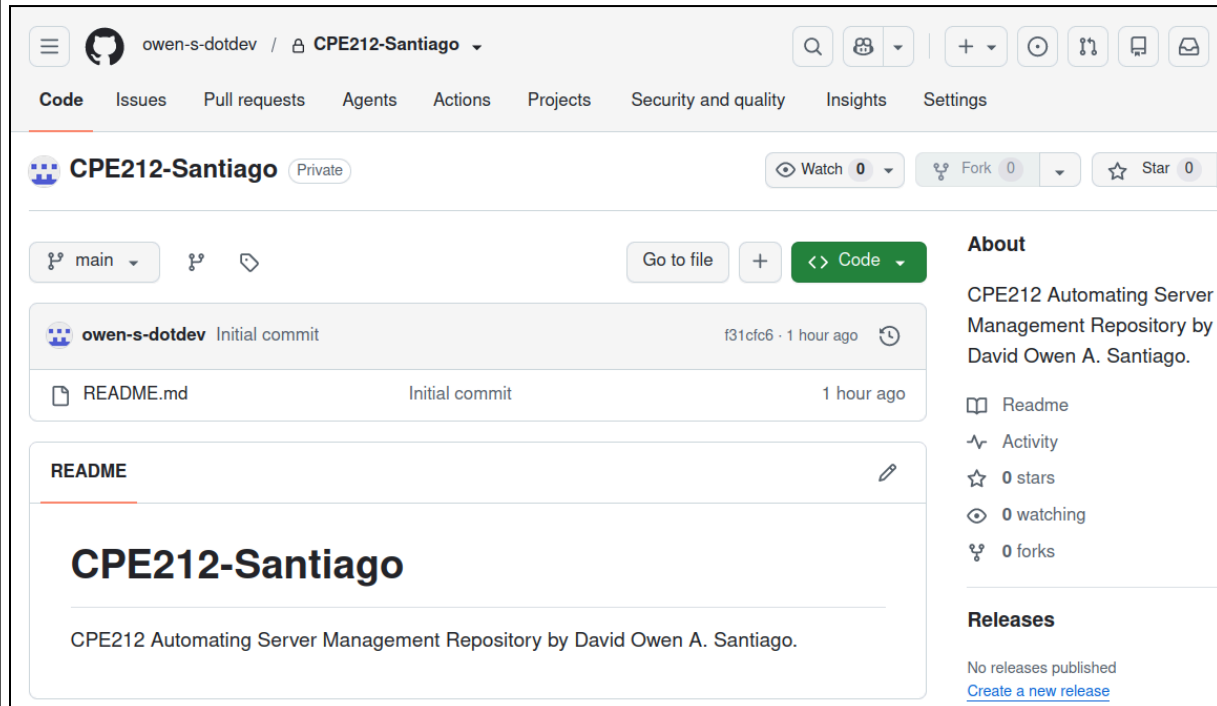
3. The version of git installed in your device is the latest. Try issuing the command *git --version* to know the version installed.

```
workstation@workstation:~$ git --version
git version 2.43.0
workstation@workstation:~$
```

4. Using the browser in the local machine, go to www.github.com.

5. Sign up in case you don't have an account yet. Otherwise, login to your GitHub account.

- Create a new repository and name it as CPE232_yourname. Check Add a README file and click Create repository.



The screenshot shows the GitHub interface for a repository named 'CPE212-Santiago' owned by 'owen-s-dotdev'. The repository is marked as 'Private'. It shows an initial commit by 'owen-s-dotdev' 1 hour ago, with a file named 'README.md' also committed 1 hour ago. The README content is visible, titled 'CPE212-Santiago', with a description: 'CPE212 Automating Server Management Repository by David Owen A. Santiago.' The right sidebar shows repository statistics: 0 stars, 0 watching, and 0 forks. There are no releases published.

- Create a new SSH key on GitHub. Go to your profile's settings and click SSH and GPG keys. If there is an existing key, make sure to delete it. To create new SSH keys, click New SSH Key. Write CPE232 key as the title of the key.

Add new SSH Key

Title

CPE212

Key type

Authentication Key

Key

- c. On the local machine's terminal, issue the command `cat .ssh/id_rsa.pub` and copy the public key. Paste it on the GitHub key and press Add SSH key.

Add new SSH Key

Title

CPE212

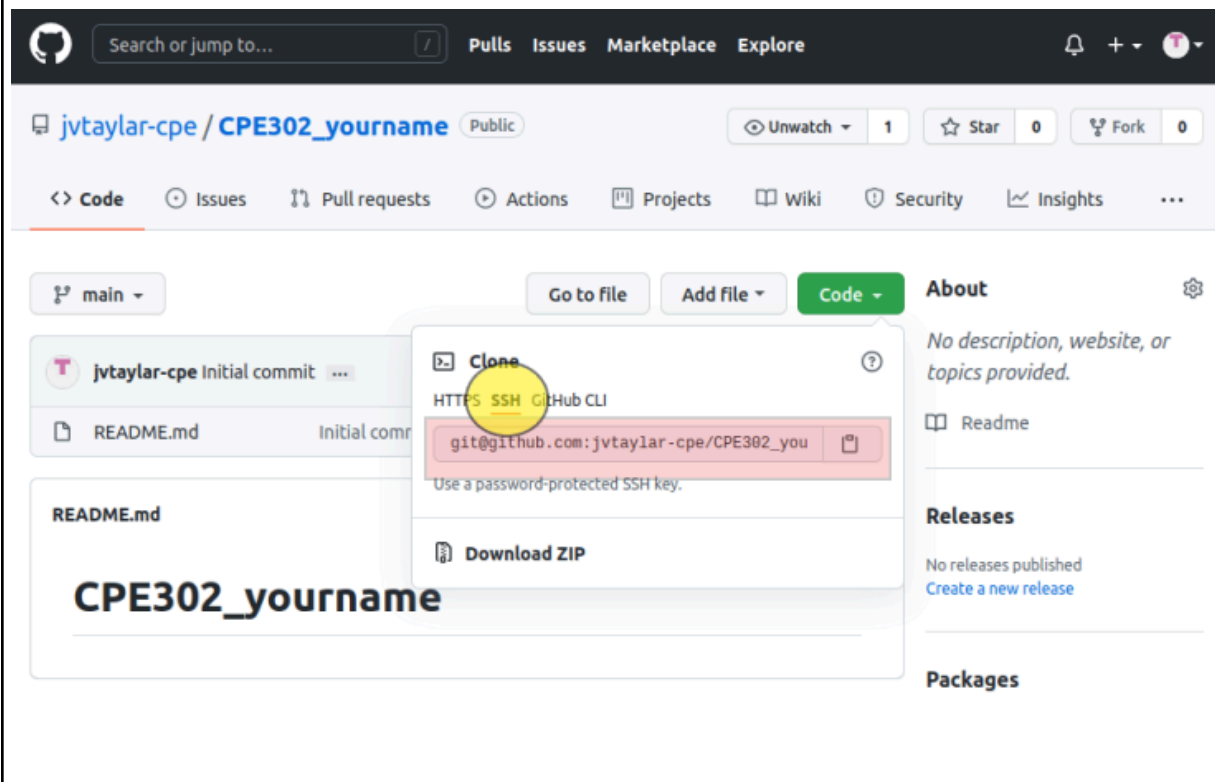
Key type

Authentication Key

Key

ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQC28e6kK13iQQU/Wwdj1rUgP7YKGbQC3N/pJniiQ55RPc2PKPWsFT0f4NQ24EhchwelkVuHS+rD8M3Q+Fqcqe9z9x4GI8eBwcwVBBY2hWsWw0

- d. Clone the repository that you created. In doing this, you need to get the link from GitHub. Browse to your repository as shown below. Click on the Code drop down menu. Select SSH and copy the link.



The screenshot shows the GitHub interface for a repository named 'CPE302_yourname' by user 'jvtaylor-cpe'. The repository is public and has 1 pull request, 0 stars, and 0 forks. The 'Code' button is highlighted, and a dropdown menu is open, showing options to 'Clone' (via SSH or GitHub CLI), 'Download ZIP', and 'Use a password-protected SSH key'. The repository name 'CPE302_yourname' is visible in the main content area.

- e. Issue the command `git clone` followed by the copied link. For example, `git clone git@github.com:jvtaylor-cpe/CPE232_yourname.git`. When prompted to continue connecting, type yes and press enter.

```
workstation@workstation:~$ git clone git@github.com:owen-s-dotdev/CPE212-Santiago.o.git
Cloning into 'CPE212-Santiago'...
The authenticity of host 'github.com (20.205.243.166)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvV6TuJJhbPZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
workstation@workstation:~$
```

- f. To verify that you have cloned the GitHub repository, issue the command `ls`. Observe that you have the `CPE232_yourname` in the list of your directories. Use `CD` command to go to that directory and `LS` command to see the file `README.md`.

```
workstation@workstation:~$ ls
CPE212-Santiago  Documents  Music      Public  Templates
Desktop          Downloads  Pictures   snap    Videos
workstation@workstation:~$ cd CPE212-Santiago
workstation@workstation:~/CPE212-Santiago$ ls
README.md
workstation@workstation:~/CPE212-Santiago$ cat README.md
# CPE212-Santiago
CPE212 Automating Server Management Repository by David Owen A. Santiago.
workstation@workstation:~/CPE212-Santiago$
```

- g. Use the following commands to personalize your git.

- `git config --global user.name "Your Name"`
- `git config --global user.email yourname@email.com`
- Verify that you have personalized the config file using the command `cat ~/.gitconfig`

```
workstation@workstation:~$ git config --global user.name "owensantiago"
workstation@workstation:~$ git config --global user.email owensantiago115@gmail.com
workstation@workstation:~$ cat ~/.gitconfig
[user]
  name = owensantiago
  email = owensantiago115@gmail.com
workstation@workstation:~$
```

- h. Edit the `README.md` file using `nano` command. Provide any information on the markdown file pertaining to the repository you created. Make sure to write out or save the file and exit.

```
workstation@workstation: ~/CPE212-Santiago
GNU nano 6.2 README.md *
# CPE212-Santiago
CPE212 Automating Server Management Repository by David Owen A. Santiago.
This repository contains the activities, projects, and portfolio in System Administration 2.
```

- i. Use the *git status* command to display the state of the working directory and the staging area. This command shows which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show any information regarding the committed project history. What is the result of issuing this command?

```
workstation@workstation:~/CPE212-Santiago$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
workstation@workstation:~/CPE212-Santiago$
```

- j. Use the command *git add README.md* to add the file into the staging area.

```
workstation@workstation:~/CPE212-Santiago$ git add README.md
workstation@workstation:~/CPE212-Santiago$ git commit -m "Changed description of README.md"
```

- k. Use the *git commit -m "your message"* to create a snapshot of the staged changes along the timeline of the Git projects history. The use of this command is required to select the changes that will be staged for the next commit.

```
workstation@workstation:~/CPE212-Santiago$ git commit -m "Changed description of README.md"
[main 9f99aa9] Changed description of README.md
1 file changed, 1 insertion(+)
```

- l. Use the command *git push <remote><branch>* to upload the local repository content to GitHub repository. Pushing means to transfer commits from the local repository to the remote repository. As an example, you may issue *git push origin main*.

```
workstation@workstation:~/CPE212-Santiago$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 3 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 401 bytes | 401.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:owen-s-dotdev/CPE212-Santiago.git
f31cfc6..9f99aa9  main -> main
```

- m. On the GitHub repository, verify that the changes have been made to README.md by refreshing the page. Describe the README.md file. You can notice how long the last commit is. It should be some minutes ago and the message you typed on the git commit command should be there. Also, the README.md file should have been edited according to the text you wrote.



owen-s-dotdev Changed description of README.md

9f99aa9 · 1 minute ago



README.md

Changed description of README.md

1 minute ago

README

CPE212-Santiago

CPE212 Automating Server Management Repository by David Owen A. Santiago. This repository contains the activities, projects, and portfolio in System Administration 2.

Reflections:

Answer the following:

1. What sort of things have we so far done to the remote servers using ansible commands?

Ansible commands have not been used in this activity because it is still setting up the remote server environment. The SSH environment was set up using basic ssh commands to generate the key-pairs and establish secure connections between the servers and host machines. Meanwhile, to document and keep track of changes

with the environment, we set up git in the machine and the github repository. The setup involved installing and updating the git version in the machine and creating the relevant repository to store the changes. I set up my global git username and password and I also set up the SSH github key in order to clone the repository in the git machine. To test the environment, I modified the README.md file in the cloned repository and pushed the change to the github repository.

2. How important is the inventory file?

The inventory file in ansible is crucial because it will serve as the storage or tracker of the relevant files, users, hosts, ssh, and other machines that it will be automating. It serves as the list of hosts that the control node will be managing and is stored in a /hosts file. It is not limited to simple hostnames and ip addresses; it provides powerful tools for automating and managing systems.

Reference:

GeeksforGeeks. (2025, August 12). *Overview of ansible inventories*. GeeksforGeeks.
<https://www.geeksforgeeks.org/devops/overview-of-ansible-inventories/>

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. CONCLUSION/LEARNINGS

In conclusion, this activity continues the environment setup for server automation. After setting up SSH, I moved on to setting up key-pairs and testing their connectivity to the two server machines. This is my first time properly setting up ssh since I only partially explored it from the previous system administration course. This serves both as a refresher and as an exploration activity expanding my knowledge about the applications of SSH in a practical and hands-on perspective. Afterwards, I once again set git and github up in my system for keeping track of changes and documenting my work. This process was more seamless since I already set up git during my previous projects. Doing these simple set ups is important for automation specifically since two of the most important factors in automation are security and documentation. SSH provides a secure connection channel between machines and servers, preventing cyberattacks from compromising confidential files and systems. Meanwhile, Github is the most popular and considered industry standard for managing version control in software development. DevOps and many services use Github as their primary version control tool since it provides transparent and precise documentation with changes and commits by having different branches for each version. Repositories are containerized, which means changes that cause issues can easily be isolated and prevented from affecting the running services. For server automation, prevention of service interruption is essential for its success. With these important setups finished, I am ready for starting actual server automation.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT



COMPUTER
ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.